Sistemas de Numeração

Sistema Numeração Decimal

- Sistema decimal utiliza 10 símbolos: 0 a 9
- Sistema posicional (cada dígito possui um peso, potência de 10 - base)
- Por exemplo:

Peso

$$345{,}2 = 3 * 10^{2} + 4 * 10^{1} + 5 * 10^{0} + 2 * 10^{-1}$$

Base 10

- Genericamente, para um número $N_{(b)}=d_{k-1}\ d_{k-2} \ldots d_1\ d_0,\ d_{-1}\ d_{-m}$ em base *b*), o valor decimal, $N'_{(10)}$ é dado por:

$$N'_{(10)} = \sum_{i=-m}^{k-1} d_i b^i = d_{k-1}b^{k-1} + \cdots + d_0 b^0 + d_{-1}b^{-1} + \cdots + d_{-m}b^{-m}$$

Sistemas Numeração - Exemplo

- Consideremos o número $467{,}1_{(8)}$
- Em base 8 temos apenas 8 símbolos (0 a 7)
- O equivalente decimal é:

$$N'_{(10)} = 4 * 8^2 + 6 * 8^1 + 7 * 8^0 + 1 * 8^{-1}$$

$$= 311{,}125_{(10)}$$

Sistema Binário – Base 2

- Utiliza apenas dois dígitos 0 e 1
- Consideremos o número binário $101{,}1_{(2)}$
- O equivalente decimal é:

$$N'_{(10)} = 1 * 2^2 + 0 * 2^1 + 1 * 2^0 + 1 * 2^{-1}$$
$$= 5{,}5_{(10)}$$

- É útil conhecermos as primeiras potências de 2
- Alguns exemplos:
  - $2^{13} = 2^3 * 2^{10} = 8 * 1\ k = 8\ k$
  - $2^{16} = 2^6 * 2^{10} = 64 * 1\ k = 64\ k$
  - $2^{24} = 2^4 * 2^{20} = 16 * 1\ M = 16\ M$

n	2^n
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1024 = 1 k
16	65536 = 64 k
20	1 M = 1 k x 1 k
30	1 G = 1 k x 1 k x 1 k

Contar em Binário

Idêntico ao que se faz em decimal. Após atingir o último dígito numa posição, adiciona 1 à posição seguinte

Decimal	Binário
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

Base 16

- Utiliza 16 dígitos: 0, 1, …, 8, 9, A, B, C, D, E, F

$A_{(16)} = 10_{(10)}$

$B_{(16)} = 11_{(10)}$

$C_{(16)} = 12_{(10)}$

$D_{(16)} = 13_{(10)}$

$E_{(16)} = 14_{(10)}$

$F_{(16)} = 15_{(10)}$

Por exemplo, o número hexadecimal $A9C,8_{(16)}$ tem equivalente decimal:

$$A9C,8_{(16)} = 10 * 16^2 + 9 * 16^1 + 12 * 16^0 + 8 * 16^{-1}$$

$$= 2716,5_{(10)}$$

Conversão base 10 para outra base – parte inteira

ISEL

- A conversão da parte inteira é feita com divisões sucessivas pela base até obter um quociente nulo. O número equivalente na nova base é obtido com os restos sucessivos;
- Por exemplo, para converter $35_{(10)}$ para binário, temos:

$$
\begin{array}{rcl}
35 : 2 = 17 & \text{resto } 1 & \rightarrow d_0 \\
17 : 2 = 8 & \text{resto } 1 & \rightarrow d_1 \\
8 : 2 = 4 & \text{resto } 0 & \rightarrow d_2 \\
4 : 2 = 2 & \text{resto } 0 & \rightarrow d_3 \\
2 : 2 = 1 & \text{resto } 0 & \rightarrow d_4 \\
1 : 2 = 0 & \text{resto } 1 & \rightarrow d_5
\end{array}
$$

$$35_{(10)} = 100011_{(2)}$$

Conversão base 10 para outra base – parte inteira: Exercício

ISEL

- Converta para binário os seguintes inteiros:

$$64_{(10)} = 1000000_{(2)}$$

$$178_{(10)} = 10110010_{(2)}$$

Conversão base 10 para outra base – parte decimal

ISEL

- A conversão da parte fracionária é feita com multiplicações sucessivas, até que seja obtida a precisão pretendida ou o valor 0. A parte inteira das multiplicações sucessivas é retida e utilizada para definir o número na nova base
- Por exemplo, para converter $0{,}6875_{(10)}$ para binário, temos:

$0{,}6875 * 2 = 0{,}375$ inteiro 1 $\rightarrow d_{-1}$

$0{,}375 * 2 = 0{,}75$ inteiro 0 $\rightarrow d_{-2}$

$0{,}75 * 2 = 0{,}5$ inteiro 1 $\rightarrow d_{-3}$

$0{,}5 * 2 = 0{,}0$ inteiro 1 $\rightarrow d_{-4}$

$0{,}6875_{(10)} = 0{,}1011_{(2)}$

Conversão base 10 para outra base – parte decimal: Exercício

ISEL

- Converta para binário os seguintes decimais:

$$0{,}625_{(10)} = 0{,}101_{(2)}$$

$$0{,}178_{(10)} = 0{,}001011011\ldots_{(2)}$$

Conversão Entre Bases não Decimais

ISEL

- Em geral, utiliza-se a base decimal como auxiliar. Para converter entre a base B1 e a base B2:
  - Converter B1 para decimal
  - Converter decimal para B2
- Caso se tratem de bases potência de 2 (2, 4, 8, 16, etc.), é mais simples
- O caso mais comum é o de conversão entre base 2 e 16.

Conversão Entre Base 2 e 16

- Cada dígito hexadecimal é representado com 4 dígitos binários
- Não sobram representações
- Assim, basta formar grupos de 4 bits a partir da vírgula
- Por exemplo, para converter $111101{,}11_{(2)}$ para hexadecimal

  0011 1101 , 1100 Acrescentar zeros (a vermelho) para perfazer 4 bits

  3 D , C

Assim, $111101{,}11_{(2)} = 3D{,}C_{(16)}$

- Por exemplo, para converter $5B7{,}E_{(16)}$ para binário

  5 B 7 , E

  0101 1011 0111 , 1110 Os zeros à esquerda da parte inteira e à direita da parte fracionária (a vermelho) podem ser removidos

Assim, $5B7{,}E_{(16)} = 10110110111{,}111_{(2)}$

Conversão Entre Bases 2 e 16

ISEL

- Converta de base 2 para base 16:

$110100111,10111_{(2)} \quad = \quad 1A7,B8_{(16)}$

- Converta de base 16 para base 2:

$$E12F,1A_{(16)} \quad = \quad \underline{1110}\ \underline{0001}\ \underline{0010}\ \underline{1111},\underline{0001}\ \underline{1010}_{(2)}$$

$$E \qquad 1 \qquad 2 \qquad F \qquad 1 \qquad A$$

Códigos Binários

- São usados para representar:
  - Informação numérica;
  - Informação não numérica (Letras, etc.)
- Códigos numéricos usados para representar informação contável:
  - Por exemplo, representar os andares de um prédio. Considerando 6 andares (cave, r/c, … 4º andar), necessitamos de 6 códigos binários (bastam códigos 3 bits)
  - Código BCD (*Binary Coded Decimal*)

    P. ex., , $159{,}8_{(10)}$ = 0001 0101 1001 , 1000 $_{(BCD)}$

    1 5 9 8

Decimal	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Códigos Binários - Alfanuméricos

ISEL

Código ASCII - Código alfanumérico usado para codificar letras, símbolos de pontuação e de controlo (7 bits)

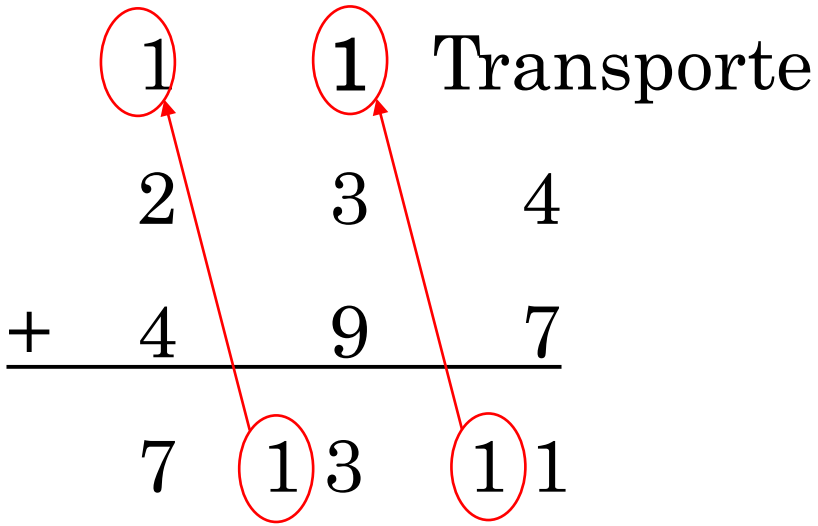
$b_3b_2b_1b_0$	$b_6b_5b_4$							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	’	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	”	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	’	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	|
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	ˆ	n	˜
1111	SI	US	/	?	O	_	0	DEL

Aritmética Binária

Adição Decimal

Recordemos o algoritmo de adição decimal:

1. Começa-se pela coluna da direita (dígitos de menor peso);
   1. Se a adição da coluna não exceder 10, o resultado não altera;
   2. Se a adição da coluna exceder 10 (base = 10), o resultado da coluna é a diferença entre o valor da soma e a base. Gera-se um transporte (*carry* = 1) para a coluna seguinte;
2. Passa-se à coluna seguinte somando os dígitos da coluna com o transporte da coluna anterior



Adição Binária

O algoritmo de adição binária é semelhante ao da adição decimal, mas com b (base) = 2.

1. Começa-se pela coluna da direita (dígitos binários de menor peso);
   1. Se a adição da coluna não exceder 2, o resultado não altera;
   2. Se a adição da coluna exceder 2 (base = 2), o resultado da coluna é a diferença entre o valor da soma e a base. Gera-se um transporte (*carry* = 1) para a coluna seguinte;
2. Passa-se à coluna seguinte somando os dígitos binários da coluna com o transporte da coluna anterior

Adição Binária

Tabela de adição binária

Adição binária

+	0	1
0	0	1
1	1	10

Sem *carry*

$$\begin{array}{r} 1 \\ +\ \ 0 \\ \hline 1 \end{array}$$

Com *carry*

$$\begin{array}{r} 1\ \ 1 \\ +\ \ \ \ 1 \\ \hline 1\ 1\ 0 \end{array}$$

Adição Binária - Exemplo

	1	1				Decimal
	1	0	1	1	->	11
+		1	1	0	->	6
1	0	0	0	1	->	17

1		1	1	1	1	1	1	1	
	1	0	1	1	1	1	0	0	1
+	1	0	1	0	0	1	1	1	1
1	0	1	1	0	0	1	0	0	0

Adição Hexadecimal

Devemos saber a adição de 2 dígitos hexadecimais – tabela de adição

$$
\begin{array}{cccc}
 & 1 & 1 & \\
 & 5 & B & 3 & 8 \\
+ & 2 & 6 & D & 5 \\
\hline
 & 8 & 2 & 0 & D
\end{array}
$$

Subtração Decimal

Algoritmo de subtração binária:

1. Começa-se pela coluna da direita (dígitos de menor peso);
   1. Se o dígito do aditivo for maior ou igual do que o do subtrativo, o resultado da subtração da coluna não se altera;
   2. Se o aditivo for menor, soma-se a base (10) ao aditivo e subtrai-se o subtrativo. Gera-se um transporte (*borrow* = 1) para a coluna seguinte;
2. Passa-se à coluna seguinte subtraindo os dígitos da coluna com o transporte da coluna anterior

Exemplo: 534 - 497

	5	13	14
-	1+4	1+9	7
	0	3	7

Subtração Binária

O algoritmo de subtração binária é igualmente semelhante ao da subtração decimal, mas com b (base) = 2.

1. Começa-se pela coluna da direita (dígitos de menor peso);
   1. Se o aditivo for maior ou igual do que o subtrativo, o resultado da subtração da coluna não se altera;
   2. Se o aditivo for menor, soma-se a base (2) ao aditivo e subtrai-se o subtrativo. Gera-se um transporte (*borrow* = 1) para a coluna seguinte;
2. Passa-se à coluna seguinte subtraindo os dígitos da coluna com o transporte da coluna anterior

Subtração Binária

$$10 - 1 = 1$$

$$\begin{array}{r} 1\ 1\ 0 \\ -\ \ 1\ \ 1 \\ \hline 0\ 1 \end{array}$$

$$\begin{array}{r} 1\ 1{+}0\ 1{+}0\ \ 1\ \ 1\ 1\ 0\ \ 1 \\ -\ \ 1{+}0\ 1{+}0\ \ 1\ \ 0\ 1{+}0\ \ 1\ \ 1 \\ \hline 0\ \ 1\ \ 1\ \ 1\ \ 0\ \ 1\ \ 0 \end{array}$$

Multiplicação Binária

Tabela de multiplicação binária

Multiplicação binária

x	0	1
0	0	0
1	0	1

```
        1 0 1 1 0
x           1 0 1
-----------------
        1 0 1 1 0
    0 0 0 0 0
1 0 1 1 0
-----------------
1 1 0 1 1 1 0
```

Multiplicação por Potências de 2

Multiplicação por 2 ($10_{(2)}$)

```
        1 0 1 1 1
x             1 0
-----------------
        0 0 0 0 0
      1 0 1 1 1
-----------------
      1 0 1 1 1 0
```

Multiplicação por 4 = 2^2 ($100_{(2)}$)

```
        1 0 1 1 1
x             1 0 0
-------------------
        0 0 0 0 0 0
      0 0 0 0 0
    1 0 1 1 1
-------------------
    1 0 1 1 1 0 0
```

Multiplicação por 2^n: acrescentamos *n* 0s à direita

$10010101_{(2)}$ * 10000 = 100101010000

(=16 = 2^4)

Divisão por Potências de 2

Divisão por 2 ($10_{(2)}$)

$10111_{(2)} = 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0$

$$10111_{(2)} : 2 = \frac{1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0}{2}$$

$$= 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 + 1 * 2^{-1}$$

$$= 1011{,}1_{(2)}$$

Divisão por 2^n: deslocamos o ponto decimal para a esquerda *n* casas binárias

$148 : 16 = 10010100_{(2)} : 10000_{(2)} = 1001{,}0100_{(2)} \quad = 9{,}25$